CO1 – ASSESSMENT TOOL 1

IT Application-Based Problem Solving

Scalability Audit of an E-Commerce Platform

Student Name: RAJASHREE S

Register Number: 192521079

Department / Class: B TECH IT _CSA0608 , DESIGN AND ANALYSIS OF ALGORITHM

1. Aim

To demonstrate a simple scalability audit by estimating how response time may change as the number of incoming requests increases.

2. Requirements

Hardware: Desktop or laptop; a basic computer is sufficient.

Operating system: Windows 10/11, macOS, or Linux.

Software: Python 3.10 or later (Python 3 is sufficient for this code).

Editor/IDE: Visual Studio Code or any text editor.

VS Code extension: Python extension by Microsoft (recommended).

Internet: Not required to run this simulation; needed only to upload files to GitHub/Classroom.

Files: scalability_audit.py and this report.

Python packages: No extra packages are required; the program uses Python's built-in features.

3. Problem Statement

An e-commerce platform may respond more slowly when many users send requests at the same time. This program models several request loads and estimates response time to illustrate when scaling may be recommended. It is an educational simulation, not a live website performance test.

4. Inputs

The program uses the following predefined input values; the user does not need to type anything when running it.

Input / variable	Value and meaning
LOAD_LEVELS	[100, 300, 500, 800, 1000] requests — simulated traffic levels
CAPACITY	800 — illustrative capacity value used in the formula
TARGET_RESPONSE_TIME	2.0 seconds — maximum target for an 'Acceptable' status
Formula	Estimated response time = 0.5 + (requests / capacity)²

These are sample assumptions chosen for learning. They are not measurements from an actual ecommerce website.

5. Algorithm / Procedure

1. Start the program.
2. Set the request levels, capacity value, and target response time.
3. For each request level, calculate the estimated response time using the formula.
4. If the estimate is at or below 2.0 seconds, print 'Acceptable'; otherwise, print 'Scaling recommended'.
5. Display the results and a note that the values are estimates.
6. Stop.

6. Source Code

```
"""
Scalability Audit of an E-Commerce Platform
Educational load simulation. It does not test a live website.
"""

import time

LOAD_LEVELS = [100, 300, 500, 800, 1000]
CAPACITY = 800
TARGET_RESPONSE_TIME = 2.0  # seconds



def estimate_response_time(requests, capacity):
    """Estimate response time using a simple illustrative model."""
return 0.5 + (requests / capacity) ** 2



def main():
    print("E-Commerce Platform Scalability Audit")
print("=" * 66)
    print(f"{'Requests':<12}{'Estimated response (s)':<26}{'Status'}")
print("-" * 66)
```

```
    for requests in LOAD_LEVELS:
        response_time = estimate_response_time(requests, CAPACITY)
status = (              "Acceptable"
            if response_time <= TARGET_RESPONSE_TIME
else "Scaling recommended"
        )
        print(f"{requests:<12}{response_time:<26.2f}{status}")


    print("-" * 66)
    print(f"Target response time: {TARGET_RESPONSE_TIME:.2f} seconds")
print("Audit simulation completed.")
    print("Note: Values are estimates, not measurements from a real website.")



if __name__ == "__main__":
main()
```

7. Screenshot 1 — Source Code

This sample screenshot-style image shows the included Python source code. For your submission, also take a real screenshot of the code opened in your own VS Code window.

Sample Screenshot 1 — Python source code

```python
"""
Scalability Audit of an E-Commerce Platform
Educational load simulation. It does not test a live website.
"""

import time

LOAD_LEVELS = [100, 300, 500, 800, 1000]
CAPACITY = 800
TARGET_RESPONSE_TIME = 2.0  # seconds


def estimate_response_time(requests, capacity):
    """Estimate response time using a simple illustrative model."""
    return 0.5 + (requests / capacity) ** 2


def main():
    print("E-Commerce Platform Scalability Audit")
    print("=" * 66)
    print(f"{'Requests':<12}{'Estimated response (s)':<26}{'Status'}")
    print("-" * 66)

    for requests in LOAD_LEVELS:
        response_time = estimate_response_time(requests, CAPACITY)
        status = (
            "Acceptable"
            if response_time <= TARGET_RESPONSE_TIME
            else "Scaling recommended"
        )
        print(f"{requests:<12}{response_time:<26.2f}{status}")

    print("-" * 66)
    print(f"Target response time: {TARGET_RESPONSE_TIME:.2f} seconds")
    print("Audit simulation completed.")
    print("Note: Values are estimates, not measurements from a real website.")


if __name__ == "__main__":
    main()
```

Illustrative code view generated from the included .py file; run it in VS Code for your own screenshot.

8. Output

Expected output when running `python scalability_audit.py`:

E-Commerce Platform Scalability Audit

==========

Requests Estimated response (s) Status

---

100 0.52 Acceptable

300 0.64 Acceptable

500 0.89 Acceptable

800 1.50 Acceptable

1000 2.06 Scaling recommended

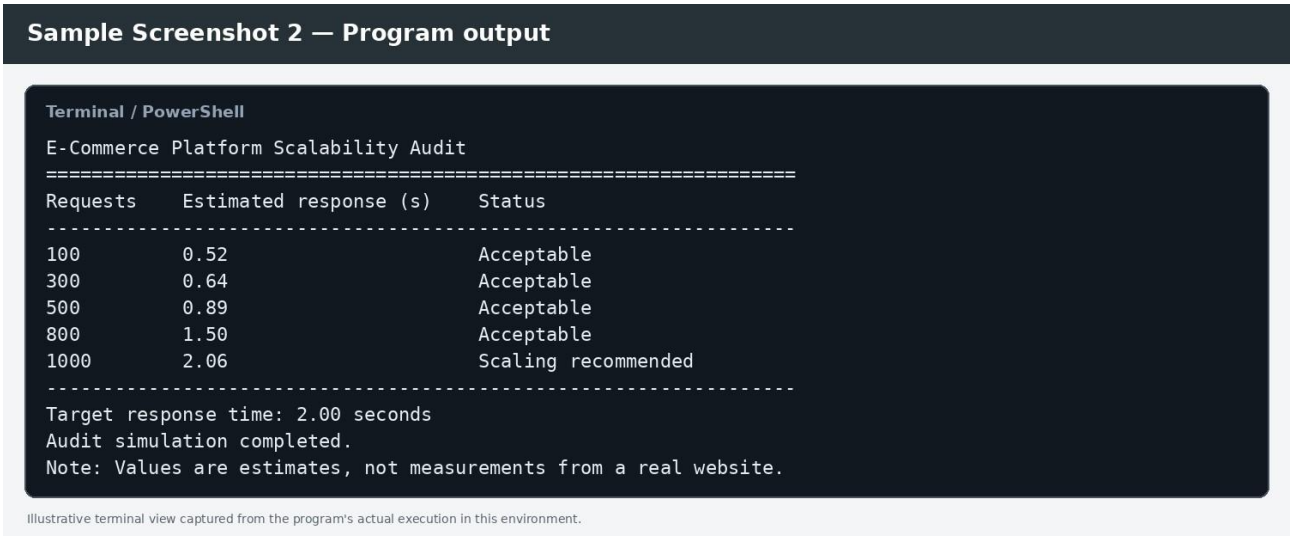
Target response time: 2.00 seconds

Audit simulation completed.

Note: Values are estimates, not measurements from a real website.

9. Screenshot 2 — Program Output

This image shows the output produced by executing the included program. You can create your own authentic screenshot by running the command in VS Code's Terminal.



10. Input and Output Summary

Requests (input)	Estimated response time (output)	Status (output)
100	0.52 seconds	Acceptable
300	0.64 seconds	Acceptable
500	0.89 seconds	Acceptable
800	1.50 seconds	Acceptable
1000	2.06 seconds	Scaling recommended

11. How to Run in VS Code (Windows)

Install Python 3 and Visual Studio Code if they are not already installed.

Open VS Code and create/open a folder named CO1_AT1/Question_1/Source_Code.

Save the provided file as scalability_audit.py inside that folder.

Open Terminal → New Terminal in VS Code.

Run: python scalability_audit.py

If the command is not recognized, try: py scalability_audit.py

When the output appears, take a screenshot using Windows + Shift + S and save it as Program_Output.png.

12. Result

The simulation estimates response times for five traffic levels. Four levels remain within the 2.0second target, while 1000 requests produces an estimated 2.06 seconds, so the program recommends scaling. These estimates illustrate the concept and do not represent real-world benchmark results.

13. Limitations and Future Improvement

This program does not send requests to a real website and does not measure actual latency, throughput, server CPU, memory, database performance, or errors. A real audit would require an authorized test environment and a load-testing tool such as Locust or JMeter, plus monitoring data.